LAPORAN PRAKTIKUM

POSTTEST 5

ALGORITMA PEMROGRAMAN DASAR

Disusun oleh:

Muhammad_Rahmat_Ramadhan 2609106029

Kelas A2 ‘26

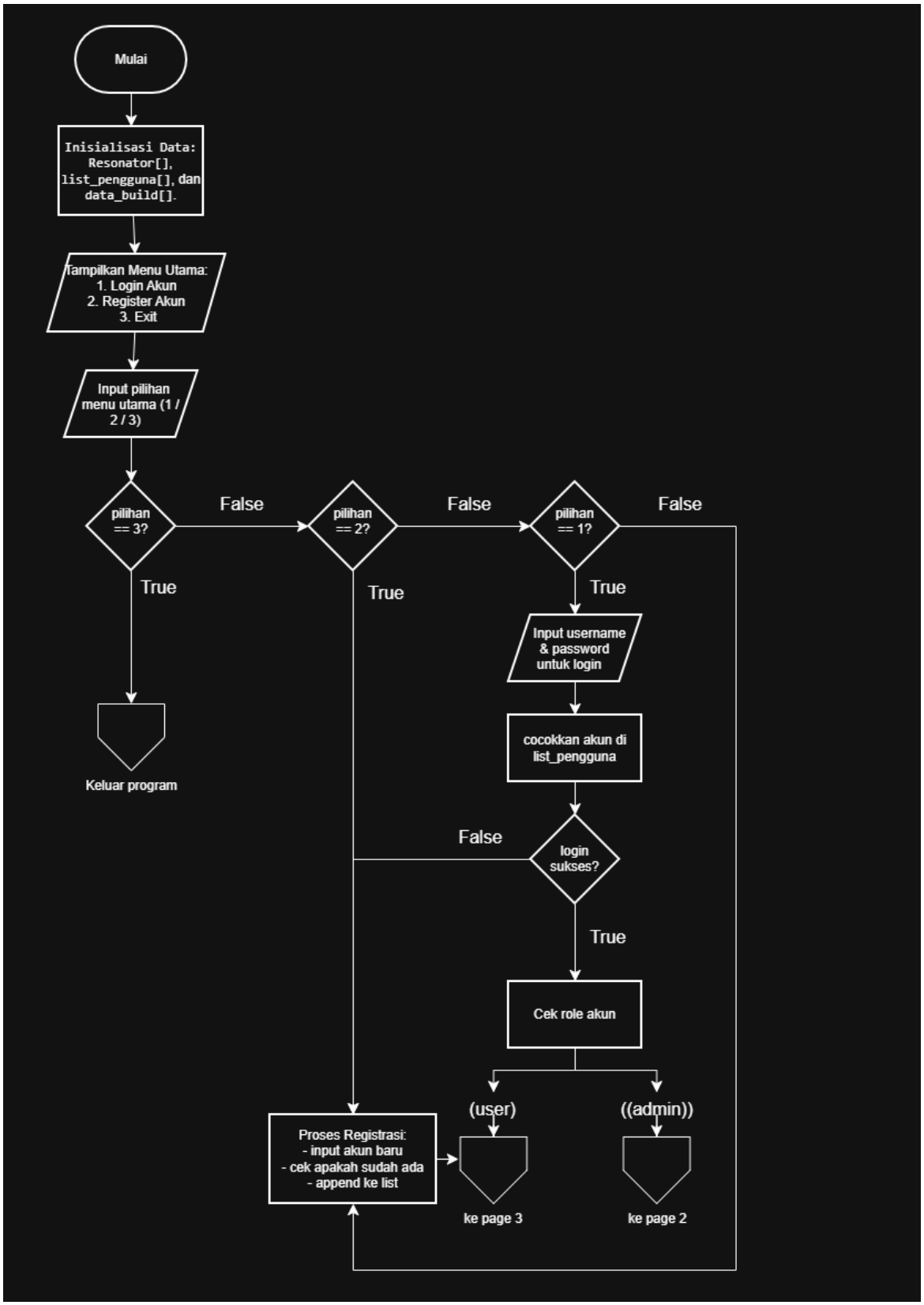
PROGRAM STUDI INFORMATIKA

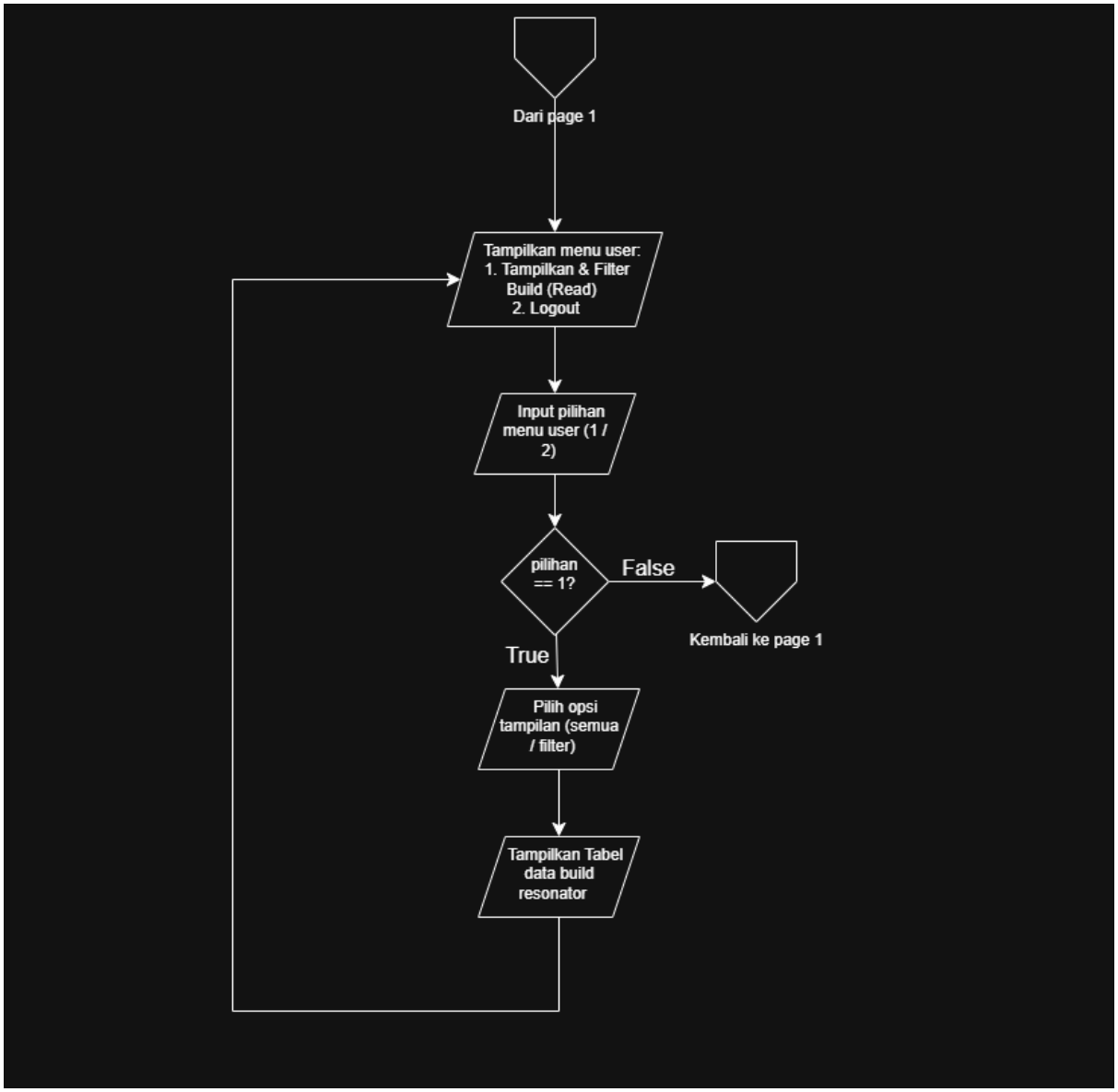
UNIVERSITAS MULAWARMAN

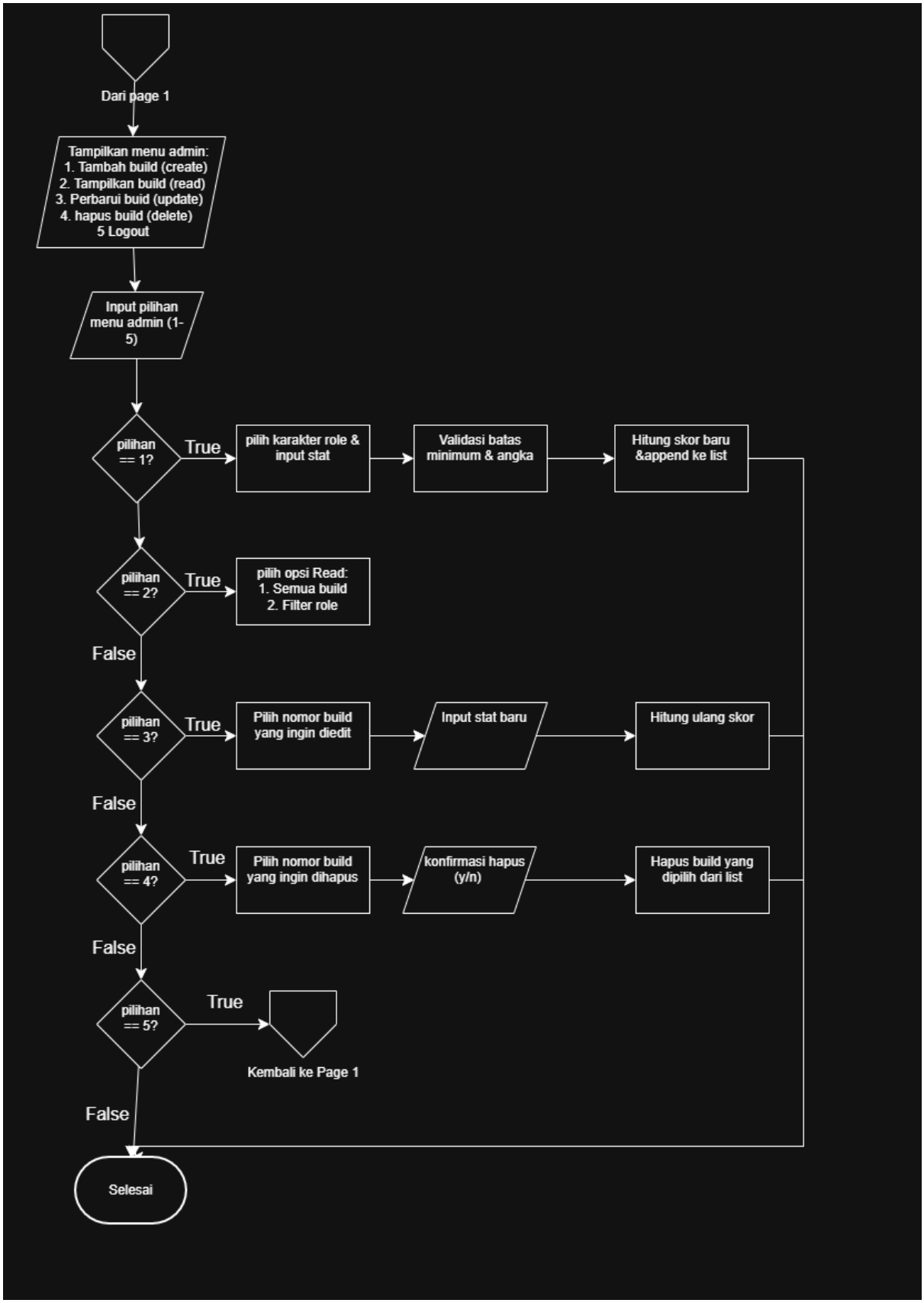
SAMARINDA

2026

1. Flowchart


Mulai
Inisialisasi Data: Resonator[], list_pengguna[], dan data_build[].
Tampilkan Menu Utama:
1. Login Akun
2. Register Akun
3. Exit
Input pilihan menu utama (1 / 2 / 3)
pilihan == 3?
False
pilihan == 2?
False
pilihan == 1?
False
True
True
True
Keluar program
Input username & password untuk login
cocokkan akun di list_pengguna
False
login sukses?
True
Cek role akun
(user)
((admin))
Proses Registrasi:
- input akun baru
- cek apakah sudah ada
- append ke list
ke page 3
ke page 2


Dari page 1
Tampilkan menu user:
1. Tampilkan & Filter Build (Read)
2. Logout
Input pilihan menu user (1 / 2)
pilihan == 1?
False
Kembali ke page 1
True
Pilih opsi tampilan (semua / filter)
Tampilkan Tabel data build resonator


Dari page 1
Tampilkan menu admin:
1. Tambah build (create)
2. Tampilkan build (read)
3. Perbarui buid (update)
4. hapus build (delete)
5 Logout
Input pilihan menu admin (1-5)
pilihan == 1?
True
pilih karakter role & input stat
Validasi batas minimum & angka
Hitung skor baru &append ke list
pilihan == 2?
True
pilih opsi Read:
1. Semua build
2. Filter role
False
pilihan == 3?
True
Pilih nomor build yang ingin diedit
Input stat baru
Hitung ulang skor
False
pilihan == 4?
True
Pilih nomor build yang ingin dihapus
konfirmasi hapus (y/n)
Hapus build yang dipilih dari list
False
pilihan == 5?
True
Kembali ke Page 1
False
Selesai

2. Deskripsi Singkat Program

Program Wuthering Waves Build & Roster Manager adalah aplikasi berbasis console Python yang dikembangkan untuk membantu pemain atau pengelola data dalam mengatur, mengevaluasi, dan menyimpan informasi build karakter (Resonator) secara terstruktur dan efisien. Program ini dirancang dengan menerapkan sistem kontrol akses berbasis peran (multi-role access control) yang membedakan hak akses antara Admin (memiliki kendali penuh CRUD: Create, Read, Update, Delete) dan User (akses khusus Read-Only untuk melihat dan memfilter data).

Dari sisi implementasi teknis, program ini memanfaatkan berbagai struktur data bawaan Python, seperti penggunaan tuple untuk master data roster karakter yang bersifat tetap, serta nested list (list bersarang) yang dinamis untuk mengelola data akun pengguna dan penyimpanan catatan build. Salah satu fitur unggulan dari program ini adalah mekanisme evaluasi build otomatis menggunakan perhitungan berbasis Crit Value serta validasi input yang ketat menggunakan perulangan while untuk memastikan nilai statistik dasar (seperti batas minimum HP, ATK, dan DEF) selalu valid sebelum data dimasukkan ke sistem.

Selain fungsionalitasnya yang komprehensif, program ini sepenuhnya mematuhi batasan aturan praktikum dasar pemrograman, yaitu murni dibangun tanpa menggunakan fungsi kustom (def), tanpa penanganan eksepsi (try-except), serta hanya mengandalkan kontrol alur standar dan modul bawaan Python untuk memastikan seluruh logika program berjalan secara sekuensial dan terstruktur

3. Source Code

```
import os

# data resonator
Resonator = ("jiyan", "yinlin", "jinshi", "changli", "zhezi", "xiangliyao", "shorekeeper", "camellya",
          "carlotta", "roccia", "phoebe", "brant", "cantarella", "zani", "ciaccona", "cartethyia", "lupa", "phrolova",
          "augusta", "iuno", "galbrena", "qiuyuan", "chisa", "lynae", "mornye", "aemeath", "luuk herssen", "sigrika",
          "hiyuki", "denia", "lucy", "rebecca", "lucilla", "yangyang: xuangling", "suisui", "qingxiao", "jingran", "hsin",
          "suoming")

# data pengguna
list_pengguna = [
    ["rahmat", "rahmat029", "Admin"],
    ["mat", "mat029", "User"]
]

# data build resonator
data_build = [
    ["Hsin", "DPS", 90, 20000, 2500, 1500, 50.0, 200.0, 300.0],
    ["Shorekeeper", "Support", 80, 15000, 1200, 1300, 15.0, 150.0, 240.0],
    ["Iuno", "Sub-DPS", 75, 13000, 1800, 900, 40.0, 160.0, 110.0],
]

# tampilan menu utama
menu_utama = True

while menu_utama:
    os.system("cls" if os.name == "nt" else "clear")
    print("=" * 60)
    print("     SISTEM MANEJEMEN BUILD RESONATOR WUTHERING WAVES    ")
    print("=" * 60)
    print("1. Login Akun")
    print("2. Registrasi Akun")
    print("3. Exit")
    print("=" * 60)
    pilihan = input("Masukkan pilihan (1/2/3): ")

    while pilihan not in ("1", "2", "3"):
        print("Pilihan tidak valid. Silakan masukkan angka 1, 2, atau 3.")
        pilihan = input("Masukkan pilihan (1/2/3): ")

    # Registrasi Akun
    if pilihan == "2":
        os.system("cls" if os.name == "nt" else "clear")
        print("=" * 60)
        print("     REGISTRASI AKUN BARU     ")
        print("=" * 60)
        username_baru = input("Masukkan username: ")

        sudah_ada = False
        for akun in list_pengguna:
            if akun[0] == username_baru:
                sudah_ada = True
                break

        if sudah_ada:
            print("[GAGAL] Username tersebut sudah terpakai. Silahkan gunakan nama lain.")
        else:
            password_baru = input("Masukkan password: ")
            list_pengguna.append([username_baru, password_baru, "User"])
```

```
        print("[BERHASIL] Akun berhasil didaftarkan. Silahkan login untuk melanjutkan.")
    input("Tekan Enter untuk kembali ke menu utama...")

elif pilihan == "1":
    os.system("cls" if os.name == "nt" else "clear")
    print("=" * 60)
    print("     LOGIN AKUN     ")
    print("=" * 60)
    username_login = input("Masukkan username: ")
    password_login = input("Masukkan password: ")

    role_pengguna = ""
    login_berhasil = False
    for akun in list_pengguna:
        if akun[0] == username_login and akun[1] == password_login:
            login_berhasil = True
            role_pengguna = akun[2]
            break

    if not login_berhasil:
        print("[GAGAL] Username atau password salah. Silahkan coba lagi.")
        input("Tekan Enter untuk kembali ke menu utama...")
    else:
        print(f"[BERHASIL] Selamat datang, {username_login}! Anda login sebagai {role_pengguna}.")
        input("Tekan Enter untuk melanjutkan...")

        # MENU ADMIN
        if role_pengguna == "Admin":
            menu_admin = True
            while menu_admin:
                os.system("cls" if os.name == "nt" else "clear")
                print("=" * 60)
                print(f"     [MENU ADMIN: {username_login}]     ")
                print("=" * 60)
                print("1. Tambah Build Resonator (Create)")
                print("2. Tampilkan Daftar Resonator (Read)")
                print("3. Perbarui Build Resonator (Update)")
                print("4. Hapus Build Resonator (Delete)")
                print("5. Logout")
                print("=" * 60)
                menu_admin_pilihan = input("Masukkan pilihan (1/2/3/4/5): ")

                # validasi input menu admin
                while menu_admin_pilihan not in ("1", "2", "3", "4", "5"):
                    print("Pilihan tidak valid. Silakan masukkan angka 1, 2, 3, 4, atau 5.")
                    menu_admin_pilihan = input("Masukkan pilihan (1/2/3/4/5): ")

                if menu_admin_pilihan == "1":
                    os.system("cls" if os.name == "nt" else "clear")
                    print("======== Tambah Build Resonator ========")

                    for urutan in range(len(Resonator)):
                        no_tampilan = urutan + 1
                        nama_resonator = Resonator[urutan]
                        print(f"{no_tampilan}. {nama_resonator}")

                    pilihan_char = input("\nPilih nomor karakter: ")

                    while not (1 <= int(pilihan_char) <= len(Resonator)):
```

```
    print("[ERROR] Nomor tidak ada di dalam daftar!")
    pilihan_char = input("Masukan nomor yang benar: ")

posisi_char = int(pilihan_char) - 1
nama_char = Resonator[posisi_char]

print(f"\nPilih Peran (Role) untuk {nama_char}:")
print("1. DPS")
print("2. Sub-DPS")
print("3. Support")
pilihan_role = input("Pilih role (1-3): ")

while pilihan_role not in ("1", "2", "3"):
    print("[ERROR] Pilihan role tidak valid!")
    pilihan_role = input("Pilih role (1-3): ")

role_char = "DPS"
if pilihan_role == "2":
    role_char = "Sub-DPS"
elif pilihan_role == "3":
    role_char = "Support"

print(f"\n===== Input Stat untuk {nama_char} ({role_char})  =====")

# Input Statistik angka
input_lvl = input("Level (1-90)        : ")
level_valid = False
while not level_valid:
    if input_lvl == "":
        input_lvl = input("Level tidak boleh kosong (1-90): ")
        continue
    cek_lvl = True
    for angka in input_lvl:
        if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
            cek_lvl = False
    if not cek_lvl:
        input_lvl = input("[ERROR] Harus angka! Masukkan level (1-90): ")
    elif not (1 <= int(input_lvl) <= 90):
        input_lvl = input("[ERROR] Level harus antara 1 sampai 90: ")
    else:
        level_valid = True
level = int(input_lvl)

# Validasi HP
input_hp = input("Total HP          : ")
hp_valid = False
while not hp_valid:
    if input_hp == "":
        input_hp = input("HP tidak boleh kosong: ")
        continue
    cek_hp = True
    for angka in input_hp:
        if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
            cek_hp = False
    if not cek_hp:
        input_hp = input("[ERROR] HP harus berupa angka positif!: ")
    elif int(input_hp) < 8000:
        input_hp = input("[ERROR] HP minimal 8000!: ")
    else:
```

```
        hp_valid = True
hp = int(input_hp)

# Validasi input nilai ATK
input_atk = input("Total ATK           : ")
atk_valid = False
while not atk_valid:
    if input_atk == "":
        input_atk = input("ATK tidak boleh kosong: ")
        continue
    cek_atk = True
    for angka in input_atk:
        if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
            cek_atk = False
    if not cek_atk:
        input_atk = input("[ERROR] ATK harus bernilai valid!: ")
    elif int(input_atk) < 1200:
        input_atk = input("[ERROR] ATK minimal 1200!: ")
    else:
        atk_valid = True
atk = int(input_atk)

# Validasi input nilai DEF
input_def = input("Total DEF           : ")
def_valid = False
while not def_valid:
    if input_def == "":
        input_def = input("DEF tidak boleh kosong: ")
        continue
    cek_def = True
    for angka in input_def:
        if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
            cek_def = False
    if not cek_def:
        input_def = input("[ERROR] DEF harus bernilai valid!: ")
    elif int(input_def) < 800:
        input_def = input("[ERROR] DEF minimal 800!: ")
    else:
        def_valid = True
Def = int(input_def)

crit_rate = float(input("Total Crit Rate (%)   : "))
crit_dmg = float(input("Total Crit DMG (%)    : "))
energy_regen = float(input("Total Energy Regen    : "))

# Perhitungan Build
crit_value = (crit_rate * 2) + crit_dmg
skor_build = crit_value + (atk / 20) + (Def / 30) + (energy_regen * 0.5)

if skor_build > 750:
    penilaian = "[GOD Build] Build impian yang sangat sempurna!"
elif skor_build >= 550:
    penilaian = "[Epic Build] Build sudah sangat layak pakai!"
elif skor_build >= 250:
    penilaian = "[Mid Build] Build standar, masih bisa ditingkatkan."
else:
    penilaian = "[Bad Build] Build kurang, perlu farm echo ulang!"

Build_baru = [nama_char, role_char, level, hp, atk, Def, crit_rate, crit_dmg, energy_regen, skor_build, penilaian]
```

```
                data_build.append(Build_baru)

                print(f"\n[BERHASIL] Build untuk {nama_char} ({role_char}) berhasil ditambahkan!")
                print(f"Skor Build: {skor_build:.2f} | Penilaian: {penilaian}")
                input("Tekan Enter untuk kembali ke menu admin...")

            elif menu_admin_pilihan == "2":
                os.system("cls" if os.name == "nt" else "clear")
                if len(data_build) == 0:
                    print("[INFO] Belum ada build resonator yang tersimpan.")
                else:
                    print("======== Menu Tampilan Build Resonator ========")
                    print("1. Tampilkan Semua Build")
                    print("2. Filter Build Berdasarkan Role")
                    sub_menu_tampilan = input("Masukkan pilihan (1/2): ")

                    if sub_menu_tampilan == "1":
                        print("\n" + "=" * 120)
                        print(f"{'No.':<5}{'Nama Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                        print("=" * 120)
                        for i in range(len(data_build)):
                            build = data_build[i]
                            print(f"{i +
1:<5}{build[0]:<20}{build[1]:<15}{build[2]:<10}{build[3]:<10}{build[4]:<10}{build[5]:<10}{build[6]:<15}{build[7]:<15}{build[8
]:<15}")
                        print("=" * 120)
                    elif sub_menu_tampilan == "2":
                        filter_role = input("Masukkan role yang ingin difilter (DPS/Sub-DPS/Support): ")
                        print("\n" + "=" * 120)
                        print(f"{'No.':<5}{'Nama Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                        print("=" * 120)
                        found = False
                        for i in range(len(data_build)):
                            if data_build[i][1].lower() == filter_role.lower():
                                build = data_build[i]
                                print(f"{i +
1:<5}{build[0]:<20}{build[1]:<15}{build[2]:<10}{build[3]:<10}{build[4]:<10}{build[5]:<10}{build[6]:<15}{build[7]:<15}{build[8
]:<15}")
                                found = True
                        print("=" * 120)
                        if not found:
                            print(f"[INFO] Tidak ada build dengan role '{filter_role}' yang ditemukan.")
                input("Tekan Enter untuk kembali ke menu admin...")

            elif menu_admin_pilihan == "3":
                os.system("cls" if os.name == "nt" else "clear")
                if len(data_build) == 0:
                    print("[INFO] Belum ada build resonator yang tersimpan.")
                else:
                    print("======== Menu build Tersimpan ========")
                    for i in range(len(data_build)):
                        print(f"{i + 1}. {data_build[i][0]} ({data_build[i][1]})")
                    pilihan_update = input("Pilih nomor build yang ingin diperbarui: ")
                    valid_update = False
                    while not valid_update:
                        cek_update = True
                        for angka in pilihan_update:
```

```
                if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
                    cek_update = False
            if not cek_update or pilihan_update == "":
                pilihan_update = input("[ERROR] Pilihan harus berupa angka valid!: ")
            elif not (1 <= int(pilihan_update) <= len(data_build)):
                pilihan_update = input("[ERROR] Pilihan tidak ada di daftar build!: ")
            else:
                valid_update = True

        index_update = int(pilihan_update) - 1
        target = data_build[index_update]

        print(f"\n===== Update Build untuk {target[0]} =====")
        print("Tekan Enter langsung jika tidak ingin mengubah stat tertentu.")

        new_level = input(f"Level ({target[2]}): ")
        if new_level != "":
            target[2] = int(new_level)

        new_hp = input(f"Total HP ({target[3]}): ")
        if new_hp != "":
            target[3] = int(new_hp)

        new_atk = input(f"Total ATK ({target[4]}): ")
        if new_atk != "":
            target[4] = int(new_atk)

        new_def = input(f"Total DEF ({target[5]}): ")
        if new_def != "":
            target[5] = int(new_def)

        new_cr = input(f"Crit Rate (%) ({target[6]}%): ")
        if new_cr != "":
            target[6] = float(new_cr)

        new_cdm = input(f"Crit DMG (%) ({target[7]}%): ")
        if new_cdm != "":
            target[7] = float(new_cdm)

        new_er = input(f"Energy Regen ({target[8]}%): ")
        if new_er != "":
            target[8] = float(new_er)

        # Perhitungan ulang skor build setelah update
        crit_value = (target[6] * 2) + target[7]
        skor_baru = crit_value + (target[4] / 20) + (target[5] / 30) + (target[8] * 0.5)

        if skor_baru > 750:
            penilaian_baru = "[GOD Build] Build impian yang sangat sempurna!"
        elif skor_baru >= 550:
            penilaian_baru = "[Epic Build] Build sudah sangat layak pakai!"
        elif skor_baru >= 250:
            penilaian_baru = "[Mid Build] Build standar, masih bisa ditingkatkan."
        else:
            penilaian_baru = "[Bad Build] Build kurang, perlu farm echo ulang!"

    input("Tekan Enter untuk kembali ke menu admin...")

elif menu_admin_pilihan == "4":
```

```
            os.system("cls" if os.name == "nt" else "clear")
            if len(data_build) == 0:
                print("[INFO] Belum ada build resonator yang tersimpan.")
            else:
                print("======== Menu Hapus Build Resonator ========")
                for i in range(len(data_build)):
                    print(f"{i + 1}. {data_build[i][0]} ({data_build[i][1]})")
                pilihan_hapus = input("Pilih nomor build yang ingin dihapus: ")
                valid_hapus = False
                while not valid_hapus:
                    cek_hapus = True
                    for angka in pilihan_hapus:
                        if angka not in ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9"):
                            cek_hapus = False
                    if not cek_hapus or pilihan_hapus == "":
                        pilihan_hapus = input("[ERROR] Pilihan harus berupa angka valid!: ")
                    elif not (1 <= int(pilihan_hapus) <= len(data_build)):
                        pilihan_hapus = input("[ERROR] Pilihan tidak ada di daftar build!: ")
                    else:
                        valid_hapus = True

                index_hapus = int(pilihan_hapus) - 1
                target_hapus = data_build[index_hapus]
                konfirmasi = input(f"Apakah Anda yakin ingin menghapus build untuk {target_hapus[0]}? (y/n): ")
                if konfirmasi.lower() == 'y':
                    del data_build[index_hapus]
                    print(f"[BERHASIL] Build untuk {target_hapus[0]} berhasil dihapus.")
                else:
                    print("[INFO] Penghapusan dibatalkan.")
            input("Tekan Enter untuk kembali ke menu admin...")

        elif menu_admin_pilihan == "5":
            print("[INFO] Logout berhasil. Kembali ke menu utama.")
            menu_admin = False
            input("Tekan Enter untuk kembali ke menu utama...")

    elif role_pengguna == "User":
        user_aktif = True
        while user_aktif:
            os.system("cls" if os.name == "nt" else "clear")
            print("-" * 55)
            print(f"     [MENU USER: {username_login}]     ")
            print("-" * 55)
            print("1. Tampilkan Daftar Resonator (Read)")
            print("2. Logout")
            print("-" * 55)
            menu_user_pilihan = input("Masukkan pilihan (1/2): ")

            while menu_user_pilihan not in ("1", "2"):
                print("Pilihan tidak valid. Silakan masukkan angka 1 atau 2.")
                menu_user_pilihan = input("Masukkan pilihan (1/2): ")

            if menu_user_pilihan == "1":
                os.system("cls" if os.name == "nt" else "clear")
                if len(data_build) == 0:
                    print("[INFO] Belum ada build resonator yang tersimpan.")
                else:
                    print("======== Menu Tampilan Build Resonator ========")
                    print("1. Tampilkan Semua Build")
```

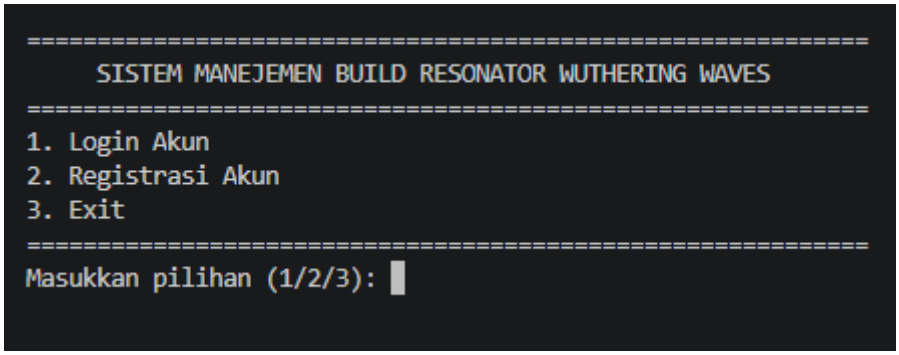
```
                    print("2. Filter Build Berdasarkan Role")
                    sub_menu_tampilan = input("Masukkan pilihan (1/2): ")

                    if sub_menu_tampilan == "1":
                        print("\n" + "=" * 100)
                        print(f"{'No.':<5}{'Nama Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                        print("=" * 100)
                        for i in range(len(data_build)):
                            build = data_build[i]
                            print(f"{i +
1:<5}{build[0]:<20}{build[1]:<15}{build[2]:<10}{build[3]:<10}{build[4]:<10}{build[5]:<10}{build[6]:<15}{build[7]:<15}{build[8
]:<15}")
                        print("=" * 100)
                    elif sub_menu_tampilan == "2":
                        filter_role = input("Masukkan role yang ingin difilter (DPS/Sub-DPS/Support): ")
                        print("\n" + "=" * 100)
                        print(f"{'No.':<5}{'Nama Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                        print("=" * 100)
                        found = False
                        for i in range(len(data_build)):
                            if data_build[i][1].lower() == filter_role.lower():
                                build = data_build[i]
                                print(f"{i +
1:<5}{build[0]:<20}{build[1]:<15}{build[2]:<10}{build[3]:<10}{build[4]:<10}{build[5]:<10}{build[6]:<15}{build[7]:<15}{build[8
]:<15}")
                                found = True
                        print("=" * 100)
                        if not found:
                            print(f"[INFO] Tidak ada build dengan role '{filter_role}' yang ditemukan.")
                input("Tekan Enter untuk kembali ke menu user...")

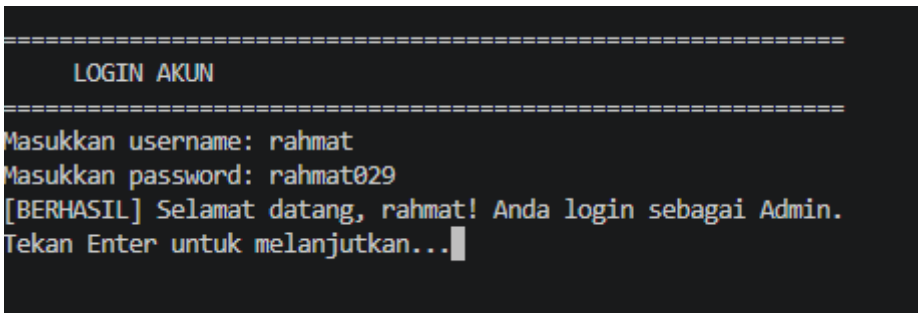
            elif menu_user_pilihan == "2":
                print("[INFO] Logout berhasil. Kembali ke menu utama.")
                user_aktif = False
                input("Tekan Enter untuk kembali ke menu utama...")

    elif pilihan == "3":
        os.system("cls" if os.name == "nt" else "clear")
        print("[INFO] Terima kasih telah menggunakan sistem ini. Sampai jumpa!")
        menu_utama = False
```

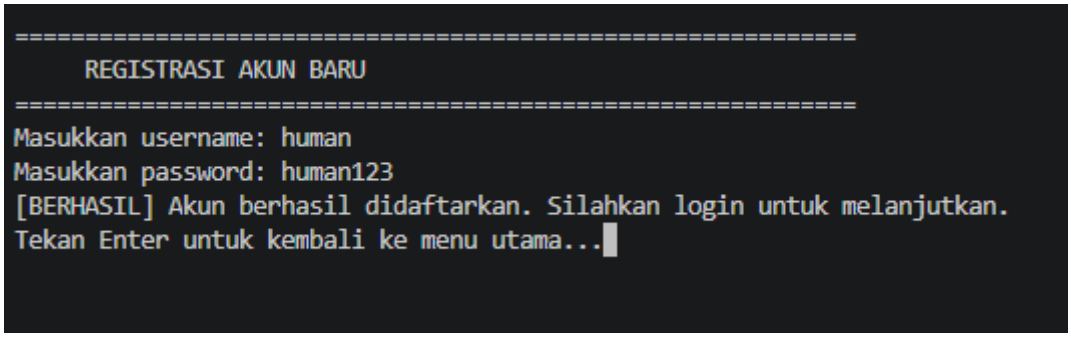
4. Hasil Output



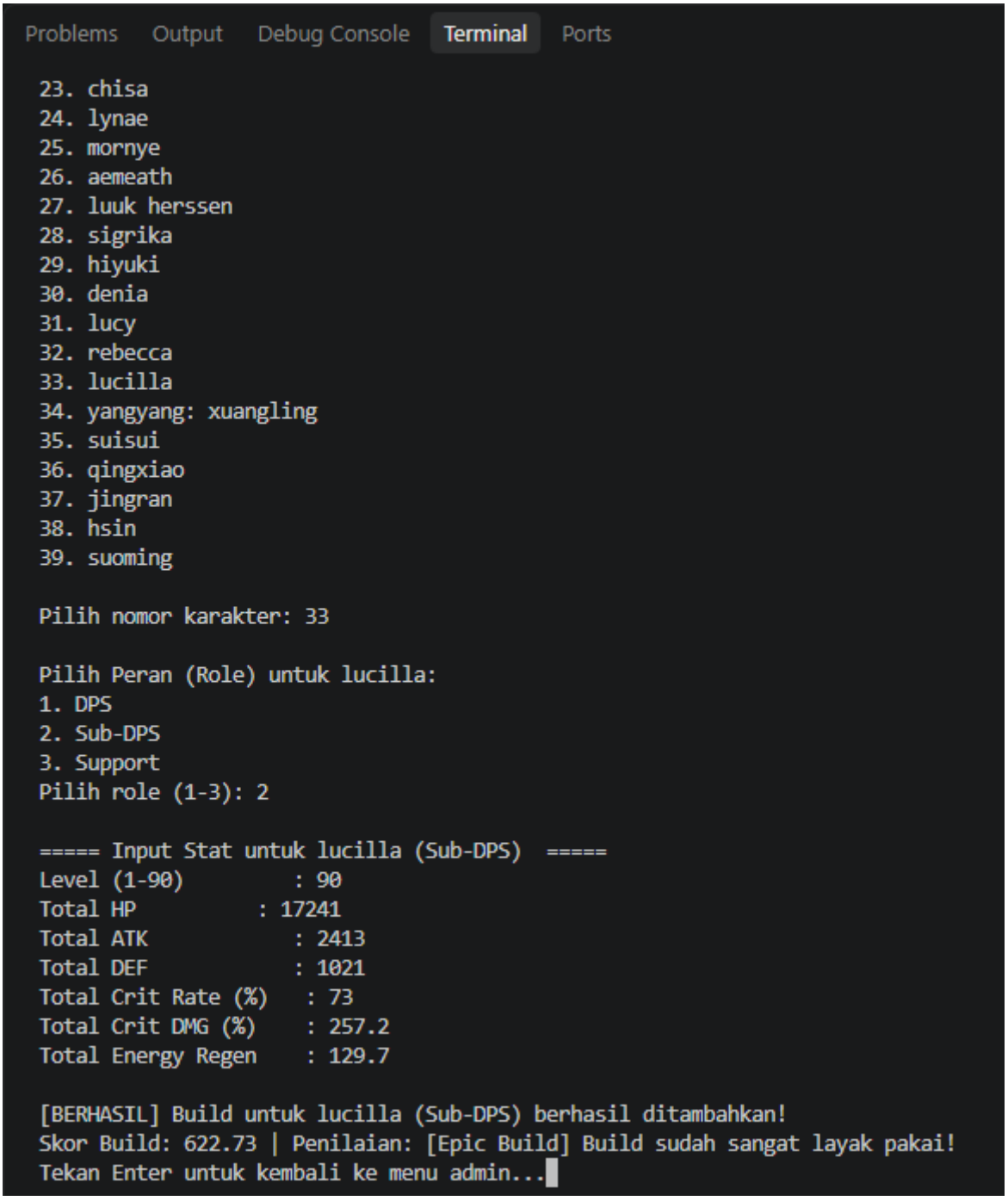
Gambar 4.1 Tampilan Menu Utama Program



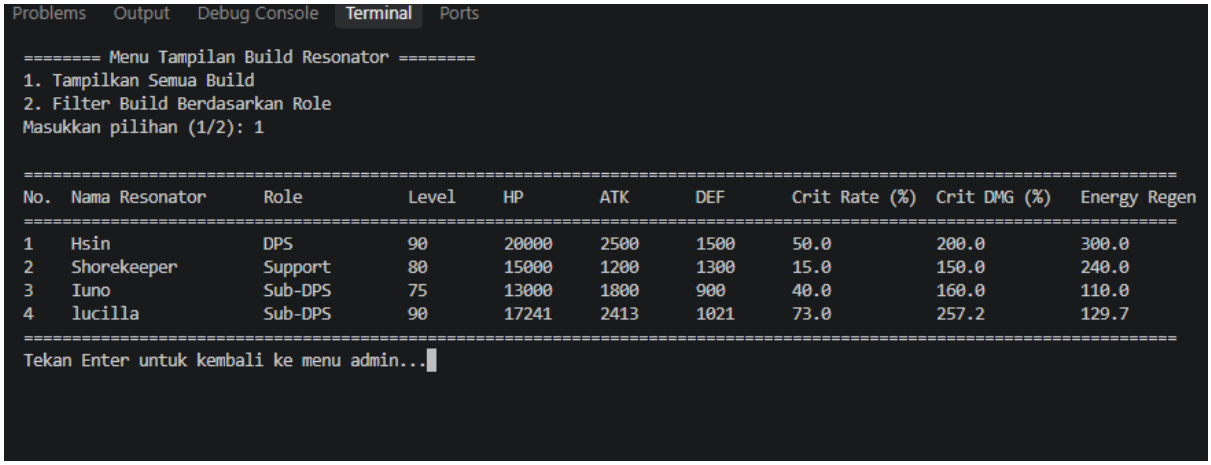
Gambar 4.2 Tampilan Menu Login Akun



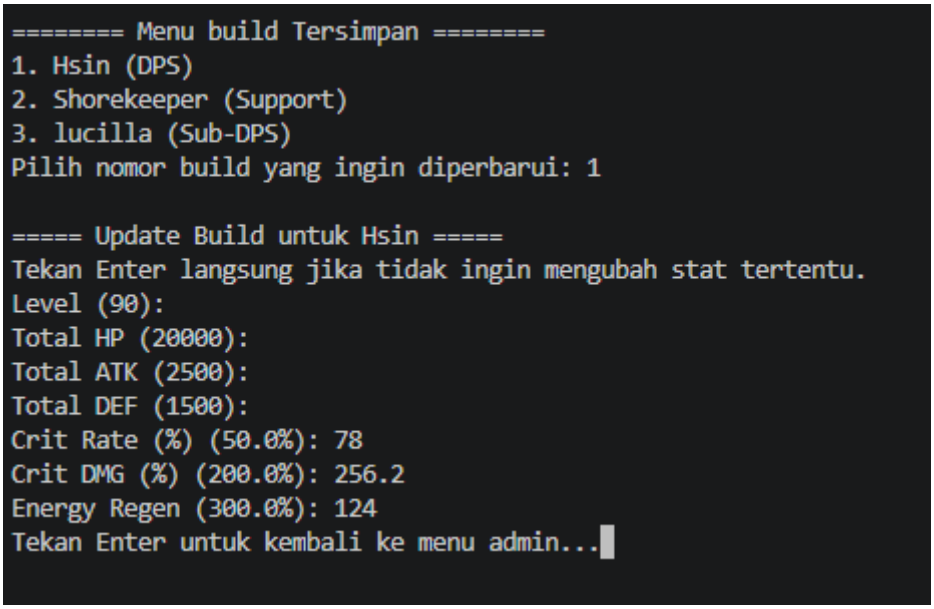
Gambar 4.3 Tampilan Menu Registrasi Akun Baru



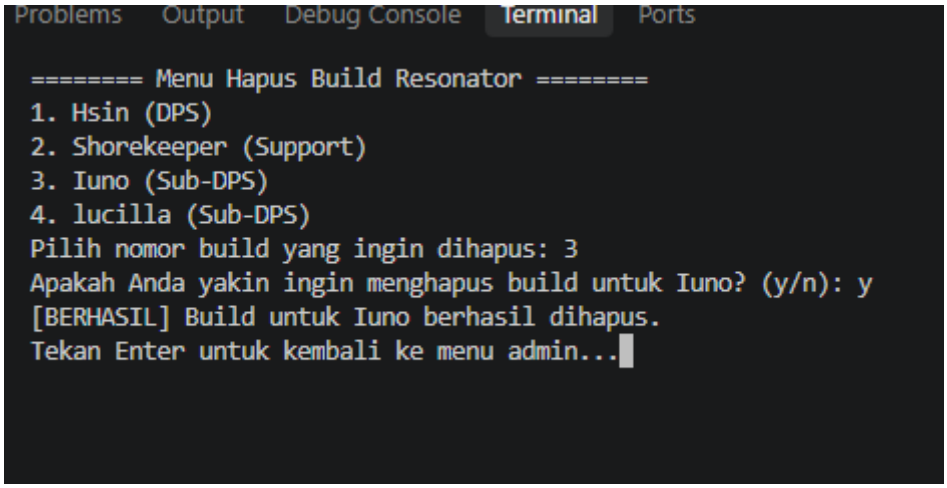
Gambar 4.4 Tampilan Menu Admin dan Fitur Create



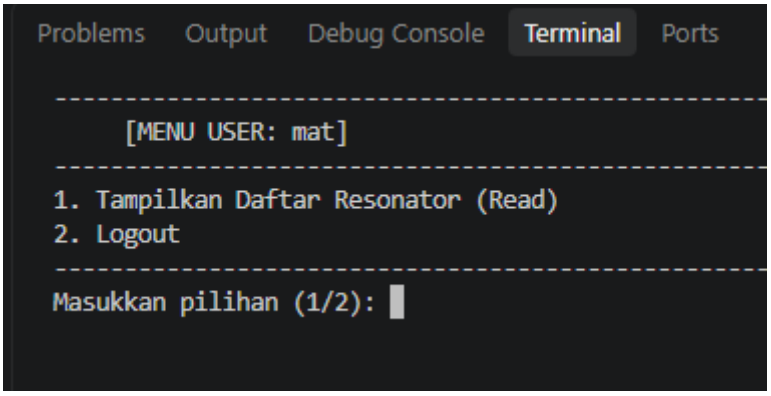
Gambar 4.5 Tampilan Output Tabel Data Build (Read)



Gambar 4.6 Tampilan Output Fitur Update



Gambar 4.7 Tampilan Output Fitur Delete

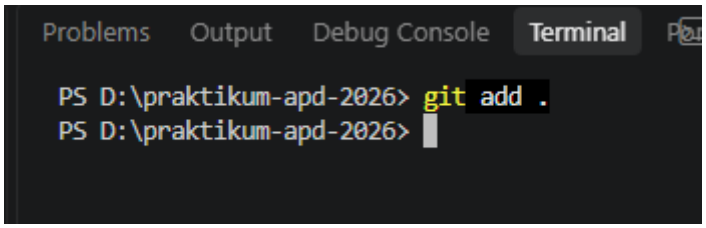


Gambar 4.8 Tampilan Akses User Reguler

5. Langkah-langkah Git

5. 1 Git Add

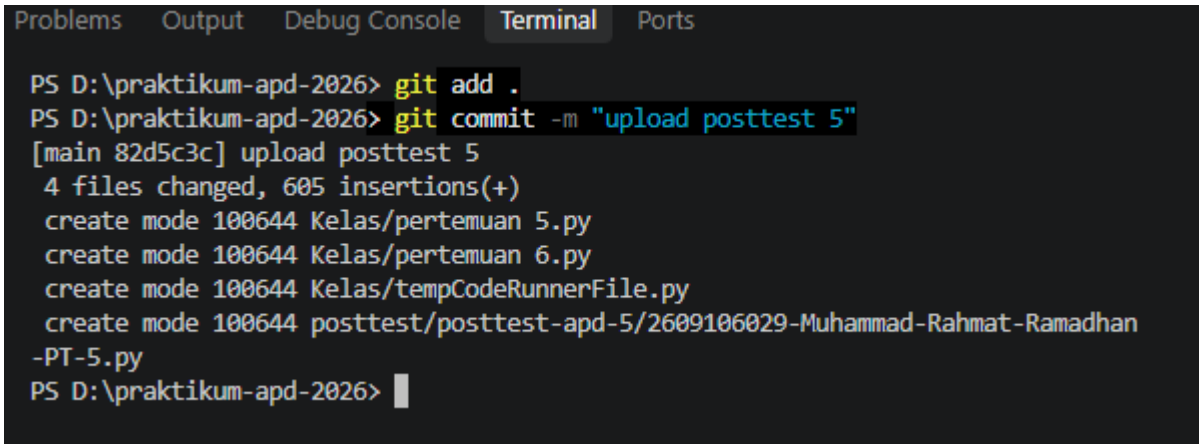
Perintah git add . digunakan untuk mendaftarkan dan menyiapkan seluruh file baru (tahap persiapan sebelum disimpan).



Gambar 5.1 Tampilan Perintah git add .

5.2 Git Commit

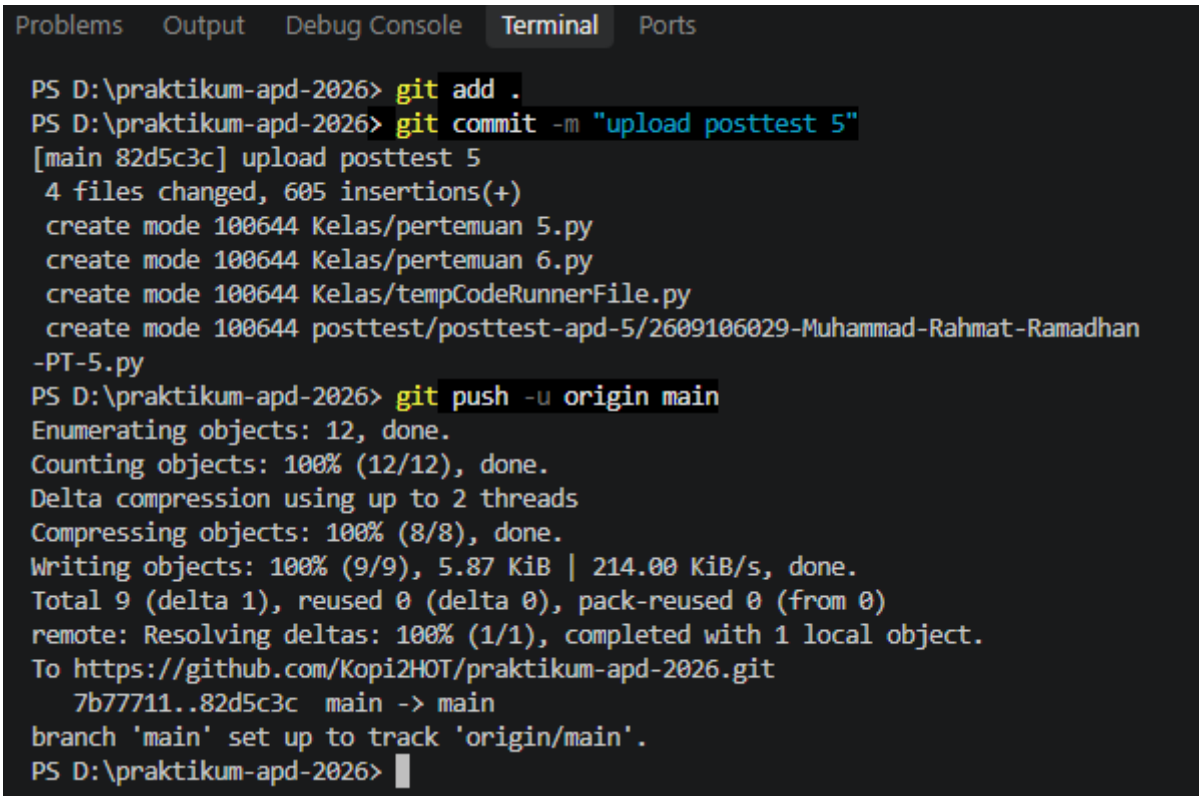
Perintah git commit -m "..." digunakan untuk menyimpan perubahan file dari staging area ke repositori lokal secara permanen, lengkap dengan pesan keterangan ringkas mengenai perubahan yang dilakukan.



Gambar 5.2 Tampilan Perintah git commit “”

5.3 Git Push

Penjelasan: Perintah git push digunakan untuk mengunggah dan menyinkronkan seluruh perubahan kode dari repositori lokal ke repositori online jarak jauh (seperti GitHub) agar tersimpan secara aman di cloud.



Gambar 5.3 Tampilan Perintah git push -u origin main